Hyperparameter Estimation: Bayesian Optimization

Piyush Bagad 150487 | **Priyadarshini Chintha** 150206 | **Siddhant Garg** 150711

1 Introduction

Hyperparameters are ubiquitous in machine learning models. Almost every machine learning model has a set of hyperparameters that need to be fine-tuned to get desirable performance of the model. For example, linear regression, arguably the simplest machine learning model has the precision of the likelihood and the prior distributions as the hyperparameters. A more modern example would be of neural networks. Several hyper-parameters like the number of layers, number of units in each of the hidden layers, learning rates for gradient descent and some regularization hyperparameters like the dropout rate etc. form a crucial component of the overall neural network architecture. It is often observed that tuning hyperparameters to their optimal values leads to substantial improvement in performance. Thus, hyper-parameter tuning is an important problem in machine learning.

1.1 Some Preliminary Solutions

- **Manual Tuning**: Manual tuning involves setting and changing hyperparameter values manually based on experience and observation. This is obviously tedious work and requires some sort of domain knowledge.
- **Grid and Random Search**: An exhaustive searching through a manually specified subset of the hyperparameter space of a learning algorithm. A grid search algorithm must be guided by some performance metric, typically measured by cross-validation on the training set or evaluation on a held-out validation set. For a random search model, given a learner $\mathcal{M}$, with parameters Θ and a loss function f, random search tries to find Θ such that f is maximized, or minimized, by evaluating f for randomly sampled values of Θ. Random search has been shown to be more effective than grid search [1].

1.2 Formal Problem Description

In a hyperparameter optimization setting, we are essentially trying to solve an optimization problem with an additional twist: the objective function is known only for a small number of points. This objective function denoted by f is called the *black-box* function.

$$f : \mathcal{X} \to \mathbb{R}$$

It is assumed that evaluation of f is expensive and since f is not known continuously, we do not have gradient information about f. One such f is the validation accuracy with a given hyperparameter setting ζ. The basic problem definition is:

$$\zeta_* = \arg\max_{\zeta \in \mathcal{X}} f(\zeta) \tag{1}$$

1.3 Automated Hyperparameter Tuning

Hyperparameter tuning can be viewed as an optimization problem: Assuming there exists some implicit mapping from the space of hyperparameters to the space of evaluation metric, most generally $\mathbb{R}$ (accuracy), our goal is to find the/a point in the hyperparameter space that will maximize/ minimize the performance metric. The approach of automated tuning relies on the inference that we are able to make considering the performance on the past set of hyperparameter configurations

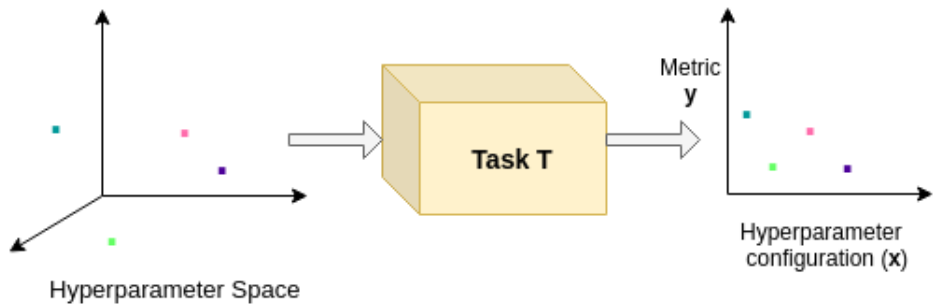


Figure 1: **Problem of hyperparameter tuning**: We have a set of points in the hyperparameter space $\mathcal{X}$ on which we know the metric value obtained through task T. The goal is to obtain that configuration of the hyperparameters which will optimize the metric.

and then using this inference to decide the choice of hyperparameters to follow next. Typically, the function mapping from the hyperparameters to the evaluation metric - a *black-box function* as it is called, may not be available as a formula, or even may be discrete. Thus, we cannot apply gradient based methods for such a optimization. Thus, a sub part of the problem involves also estimating this black-box function. Hence, the optimization is at two levels: Firstly, we are trying to achieve a reasonable estimate of the black-box function, and secondly, we are trying to reach to the optima of this blackbox function.

2 Background and Related Work

2.1 Bayesian Optimization for Hyperparameter Optimization

We assume that we have a set $\mathcal{D} := \{\zeta_n, \eta_n\}_{n=1}^N$ of observations. We assume a likelihood model with noise as

$$\eta_n \approx f(\zeta_n) \leftrightarrow \eta_n = f(\zeta_n) + \epsilon$$

for the limited amount of evaluations of f we have. We also assume a prior on the behavior of f. As usual, we infer a posterior over f and use this posterior information to find points in the search space $\mathcal{X}$ that can help us reach the optima. In order to find new points in $\mathcal{X}$, we define an *aquisition function*

$$a : \mathcal{X} \to \mathbb{R}$$

This aquisition function controls the *exploitation-exploration* trade-off. By exploitation, we mean that select new points from the region where we suspect the optima exists. For instance, if we are minimizing the black box function, then we will choose points on which the values of f would be lower. By exploration, we set out finding points in the search space where we have very little observations about the behavior of f. Once we have found a new point, we evaluate f on it and add it to the dataset $\mathcal{D}$ and update the posteriors and continue the procedure until some convergence criterion. Now, we will give a detailed account of using Bayesian optimization for choosing near-optimal hyperparameters. Let us say we have a machine learning task T at hand. We have data $\mathcal{D}_{\text{task}}$ that we can use to train and test the model for task T. Let's assume some split of the dataset.

$$\mathcal{D}_{\text{task}} = \mathcal{D}_{\text{task}}^{(\text{train})} \cup \mathcal{D}_{\text{task}}^{(\text{val})} \cup \mathcal{D}_{\text{task}}^{(\text{test})}$$

Let's say the model for T has hyperparameters $\zeta \in \mathcal{X}$ and the metric w.r.t which they are to be optimized be $\eta \in \mathbb{R}$. Assume ζ is K dimensional with its components being continuous, categorical or ordinal. The hyperparameters are related to the concerned metric by the black-box function.

$$f : \mathcal{X} \to \mathbb{R}; \;\; f(\zeta) = \eta$$

Considering the task T as an oracle which on input ζ outputs a metric value η, we now articulate the Bayesian optimization routine. Let's say we have a surrogate model which models f and we assume some prior $p(f)$ on f. Let the hyperparameters if the surrogate model and the prior together be denoted by Θ.

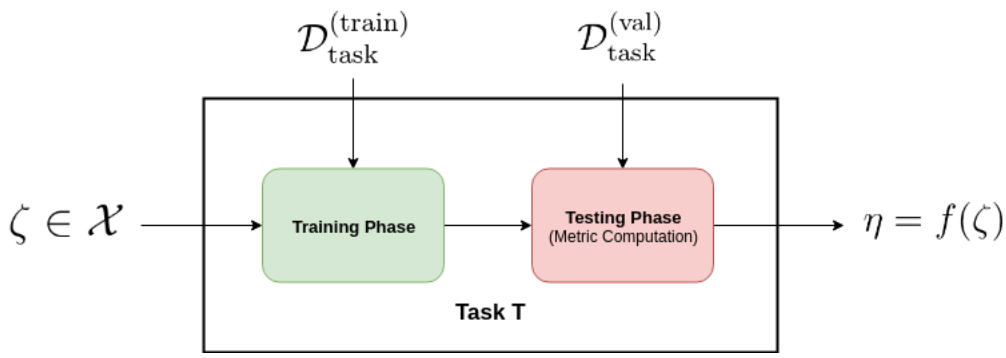


Figure 2: **Task Description**: The task T takes in the data and the hyperparameters and trains the model. The trained model is used to compute the concerned metric on the validation data.

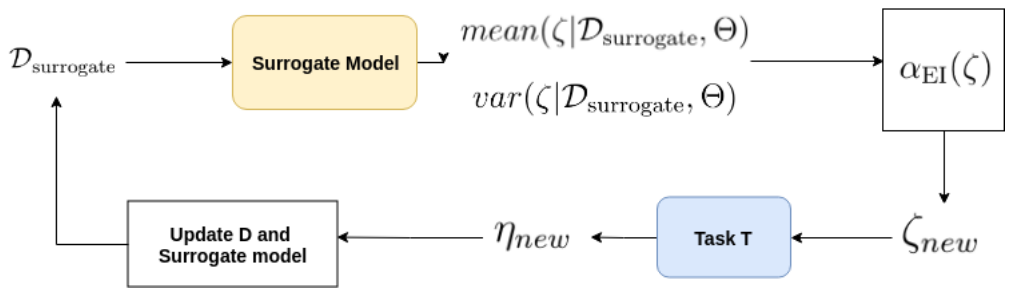


Figure 3: **Bayesian optimization routine**: Refer to section 2.1 for a detailed explanation.

- **Initialization**: Construct dataset $\mathcal{D}_{\text{surrogate}} := \{\zeta_m, \eta_m\}_{m=1}^{M}$. In other words, compute the metric for M number of hyperparameter settings.
- **Repeat until convergence**: (Can monitor some meta-metric to check convergence)
  1. Compute the posterior predictive distribution mean and variance: $mean(\zeta|\mathcal{D}_{\text{surrogate}}, \Theta)$, $var(\zeta|\mathcal{D}_{\text{surrogate}}, \Theta)$, $\forall \zeta \in \mathcal{X}$.
  2. Use the predictive mean and variance to compute the expected improvement
  $$\alpha_{\text{EI}}(\zeta) =, \forall\, \zeta \in \mathcal{X}$$
  3. Choose a new point ζ_{new} based on maximum expected improvement.
  $$\zeta_{new} = \arg\max_{\zeta \in \mathcal{X}} \alpha_{\text{EI}}(\zeta)$$
  4. Pass ζ_{new} to the task T in order to compute $\eta_{new} = f(\zeta_{new})$.
  5. Add $(\zeta_{new}, \eta_{new})$ to $\mathcal{D}_{\text{surrogate}}$. Update the posterior and continue.

2.2 Modeling Choice for Black-box Function

The properties we can extract from a Bayesian optimization model depend upon the choice of surrogate model we use to approximate the black-box function and the kind of acquisition function we use in the routine. In this section, we look at some of the choices for the surrogate model, their merits and demerits.

2.2.1 Gaussian Processes

GPs have been traditionally used as a popular model for the surrogate model due to the ease and simplicity with which posterior prediction computations can be carried out. They also provide flexibility, well-calibrated uncertainty. We assume a GP prior over the black-box function f. We get closed form posterior updates which makes GPs desirable to be used. See [4] and [3] for a rigorous treatment of using GPs in Bayesian Optimization.

2.2.2 Bayesian Neural Networks

A key limitation of GP based surrogate model is that its computational complexity scales cubicly in the number of examples. This is due to the matrix inversion of an $N \times N$ matrix during the posterior computation in GPs. This restricts the usability of GPs in a setting where large number of black-box function evaluations are necessary for reliable estimates. Snoek et al [9] introduce usage of Bayesian neural networks as a replacement to GPs to model the black-box function.

2.2.3 Probabilistic Programming

Probabilistic programming languages (PPLs) is a framework that provides a general way to model structure, problem intuition, and prior knowledge in a Bayesian model. They allow for easy model specification and composition. One of the most attractive aspects of using PPLs is that we can do automatic inference, often in the form of samples from an approximate posterior distribution in it. Neiswanger et al [6] introduce usage of PPLs to model the black-box function. They develop a BO framework in which any arbitrary PP can be used. They use an ensemble of GP, deep neural networks, mixture models, latent factor models etc. as the surrogate model.

2.3 Choice of Acquisition Function

In the Bayesian optimization routine, the choice of acquisition function to choose a new point in a sequential optimizer gives different properties. In this subsection, we compare some of the popular acquisition functions in the literature. In order to setup the notation, let's say $\mathcal{D}$ be the surrogate data, θ denote its hyperparamters and $\mathcal{X}$ be the optimization space.

2.3.1 Expected Improvement

Expected improvement [5] is one of the most popularly used method for choosing a new point in the optimization space in a sequential optimization based model.

$$\alpha_{EI}(\mathbf{x}|\mathcal{D},\theta) = \sigma(\mathbf{x}|\mathcal{D},\theta)\left(\gamma(\mathbf{x})\Phi(\mathbf{x}) + \mathcal{N}(\gamma(\mathbf{x})|0,1)\right)$$
$$\text{where } \gamma(\mathbf{x}) := \frac{f(\mathbf{x}_{best}) - \mu(\mathbf{x}|\mathcal{D},\theta)}{\sigma(\mathbf{x}|\mathcal{D},\theta)}$$

It is noted in the literature that the EI criterion tends to be too greedy (exploitative) in nature. There is a recent work [8] that improves the criterion in the direction of making it more exploratory. Even in our preliminary experiments, we observed that the vanilla EI criterion tends to exaggerate the exploitative behavior.

2.3.2 Maximum Probability Criterion

MPI is probably the first used acquisition function primarily because it is very intuitive. However, it is very rarely used in practice in the current scheme of things.

$$\alpha_{MPI}(\mathbf{x}|\mathcal{D},\theta) = p(f(\mathbf{x}) < y_{best}) = \Phi(\mathbf{x})$$

3 Scalable Bayesian Optimization

The choices of surrogate model and the acquisition function drive the kind of properties we get from a B.O. framework. Using GPs as the surrogate model along with expected improvement as the acquisition function has been a popular choice. However, it suffers from lack of scalability due to the expensive posterior computation in GPs. There have been several attempts at improving the scalability of the BO framework by considering alternate surrogate models or using multi-task setup.

3.1 Replacing GPs by Neural Nets

Replacing GPs by Bayesian neural nets was proposed by [9]. They use MAP estimation for the rest of the layers and derive Bayesian linear regression on the last layer outputs if the neural network. They show the efficacy of their framework on various real-world tasks. We considered studying this paper in detail and implementing a simplified version of it.

3.2 Multi-task Bayesian Optimization

One of the ways of alleviating the scalability issues for GP based BO framework is using it in a multi-task setup. [10] extend multi-task GPs to multi-task BO. [7] apply similar extension for Bayesian neural nets.

4 Our Work

4.1 Understanding the Domain & Implementation

We used [9] as primary reference and tried to understand the domain of Bayesian optimization in general. In order to consolidate our understanding, we implemented a simplified version of the neural-net based BO routine of the paper. We made two simplifications: we use MLE-II for estimation of the hyperparameters of the Bayesian linear regression on the last layer of the neural net and we used vanilla expected improvement criterion. The paper uses slice sampling based method for both of these. Please refer to the section 5 for the experimental setup and results we obtained using our own implementation.

4.2 Exploring possible improvements

We tried to find potential ways of improving on the papers we read. We have a decent base to build upon. Please refer to the section 6 on some of our ideas for improvements/ extensions to the papers we consider:

1. Scalable Bayesian Optimization using Deep Neural Networks [9]
2. Scalable Hyperparameter Transfer Learning [7]

4.3 Description of softwares used

We used `pytorch` as our base implementation framework. We have borrowed a simple implementation of Bayesian neural network from `pybnn` as our base model on which we built the Bayesian optimization routine.

5 Experimentation

5.1 Task and Dataset Description

We consider a set of three simple tasks. The task 0 serves the purpose of illustrating our implementation of Bayesian optimization using adaptive basis linear regression (ABLRs) (using neural nets) on a simple mathematical dataset. Figure 4 shows the dataset and figure 5 shows the results on this task. Next, we consider two simple tasks to test the efficacy of the framework to estimate

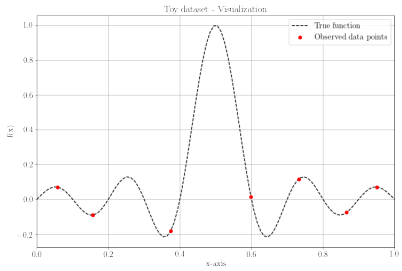
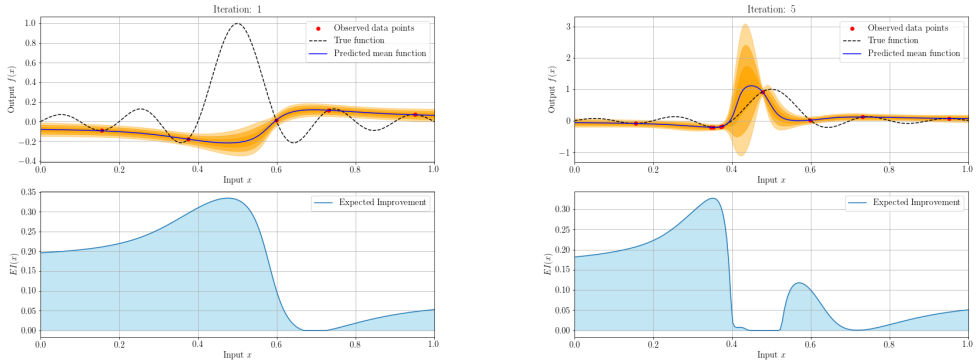


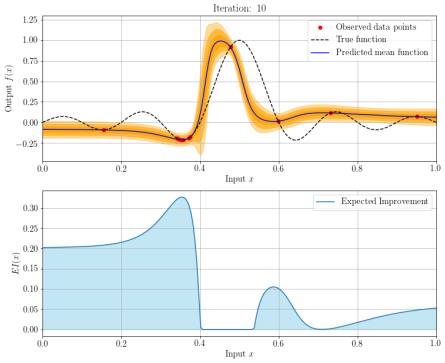
Figure 4: Toy-dataset for Task 0: $y = sinc(x),\ x \in [0, 1]$

hyperparameters.

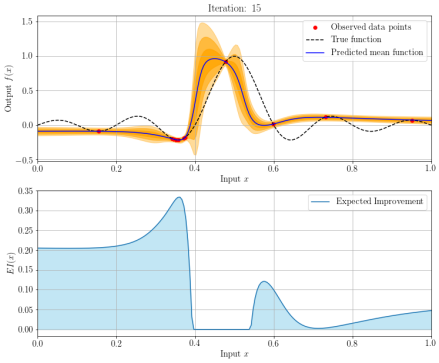


(a)

(b)



(c)



(d)

Figure 5: The ABLR-BO setting learns a crude approximation of the given black box function f. The top plot in each of the figures shows the approximate function predictive mean and variance levels while the bottom plot shows the aquisition function over the space $\mathcal{X} = [0, 1]$

.

- **Task 1: Bayesian Linear Regression on a toy-dataset**
  We consider a toy-dataset and the task is to learn a Bayesian linear regressor on a hard-coded kernel on the one-dimensional dataset. The dataset consists of 10 data points and was provided as a part of assignment 1 of the course. Please refer to assignment 1 for more details about the dataset and the task. For the purpose of this project, we consider estimation of the following hyperparameters: K: kernel dimension, λ: linear regression regularization hyperparameter and β: noise precision. We consider a fixed range of values for each of the parameters and discretize the space. The optimization of aquisition function operates in this discretized space.
- **Task 2: Logisitc Regression on MNIST dataset**
  In order to show the efficacy of the framework on a real-world dataset, we consider the task of logisitc regression on MNIST image dataset. This is one of the simplest benchmark tasks considered in the literature of hyperparameter optimization [2]. We consider a split of 50k/10k/10k for train/validation/test. For simplicity, we only consider two hyperparameters, C: regularization hyperparameter, l: intercept-scaling parameter of the `liblinear` package. We use only these two as against the four used in relevant literature because these are readily available with the `scikit-learn` logisitic regression package. However, our code is flexible enough to incorporate other hyperparameters.

5.2 Results

- **Task 1**: We obtained $K = 3, \beta = 4.0, \lambda = 1.0$ as the optimal hyperparameter configuration. This was in accordance with our Assignment 1 results. The visualization is shown in figure 6.

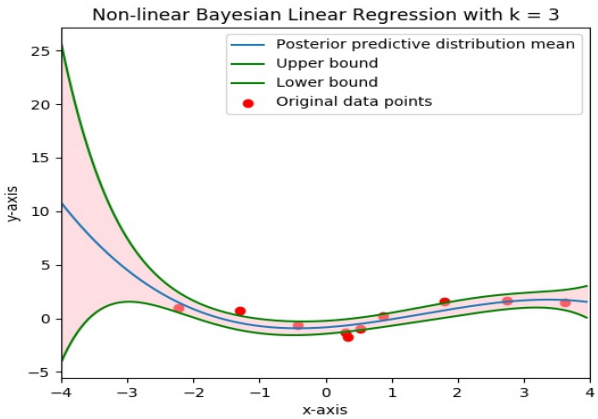


Figure 6: Visualization of results with obtained optimal hyperparameter setting on task 1
.

# Evals	Method	Test error
100	SMAC	8.60
100	TPE	8.30
100	Spearmint	**6.88**
100	DNGO	6.89
10	Our implementation	8.49

Table 1: Values for SMAC, TPE, Spearmint, DNGO taken from Snoek et al [9]. Ideally, our results should have been identical or close to the DNGO paper values. However, there were a couple of simplicifications we had assumed in our implementation: the estimation of hyperparameters of the ABLR was done using MLE-II instead of Slice sampling as suggested in the paper and the aquisition function we used was the vanilla version of expected improvement whereas they use a modified slice-sampling based expected improvement criterion.

- **Task 2**: We obtained $\{C, l\} = \{3.775, 1.625\}$ as the optimal setting on which we obtained 91.51% accuracy on the test set. The comparison with relevant literature is shown in the table 1.

6 Discussion and Future Work

Hyperparameters are ubiquitous and require careful tuning for better performance of most machine learning task. Bayesian optimization is an effective way of selecting locally optimal setting of hyperparameters. The choices of surrogate model and the acquisition function drive the kind of properties we get from a B.O. framework. Using GPs as the surrogate model along with expected improvement as the acquisition function has been a popular choice. However, it suffers from lack of scalability due to the expensive posterior computation in GPs. Replacing GPs by Bayesian neural nets was proposed by [9]. They use MAP estimation for the rest of the layers and derive Bayesian linear regression on the last layer outputs if the neural network. They show the efficacy of their framework on various real-world tasks. We considered studying this paper in detail and implementing a simplified version of it. We also considered an alternative approach which considers the surrogate model to be probabilistic programming [6]. We also studied some extentions of the BO setup to a multi-task setting. [10] extend multi-task GPs to multi-task BO. [7] apply similar extension for Bayesian neural nets. Finally, we also considered some of the possible extensions/improvements to the Bayesian optimization setup.

6.1 Scaling Up Posterior Updates in Neural Nets

In the current scenario, in each epoch, when we add a new point $(\zeta_{new}, \eta_{new})$ to $\mathcal{D}_{surrogate}$, we retrain the entire neural network to update the posterior distribution. We can think of using an online setting like that of SGLD (Stochastic gradient Langevin Dynamics) in order to have efficient posterior updates.

6.2 Generative Model for Black-box Function

In each epoch, we need to pass ζ_{new} to the task to obtain η_{new}. This can be expensive if the task is expensive, for instance a deep learning task can take up to days to compute a metric. One of the directions we though of was having a generative model as the surrogate model rather than a discriminative one. We would require a modified acqusition function that can incorporate the uncertainty in the relation of the generated two-tuple $(\zeta_{new}, \eta_{new})$. We have not thought through about this strategy at this stage and would certainly want to concretize the idea.

6.3 Learnings from the Projects

Firstly, we believe we understand the domain of Bayesian optimization well. This has helped us with the ability to skim through the papers of the domain quickly and still understand the gist of it. The papers (of this domain) do not seem intimidating. Secondly, implementing every step in the BO routine helped us understand the finer details well. Finally, we think we have a good base to try out some novel approach/improvement and perform experiments fairly quickly.

References

[1] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.*, 13:281–305, February 2012.

[2] Katharina Eggensperger, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Efficient benchmarking of hyperparameter optimizers via surrogates. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, AAAI'15, pages 1114–1120. AAAI Press, 2015.

[3] R. Garnett, M. A. Osborne, and S. J. Roberts. Bayesian optimization for sensor set selection. In *Proceedings of the 9th ACM/IEEE International Conference on Information Processing in Sensor Networks*, IPSN '10, pages 209–219, New York, NY, USA, 2010. ACM.

[4] Donald R. Jones. A taxonomy of global optimization methods based on response surfaces. *Journal of Global Optimization*, 21(4):345–383, Dec 2001.

[5] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *J. of Global Optimization*, 13(4):455–492, December 1998.

[6] Willie Neiswanger, Kirthevasan Kandasamy, Barnabás Póczos, Jeff Schneider, and Eric Xing. Probo: a framework for using probabilistic programming in bayesian optimization. *CoRR*, abs/1901.11515, 2019.

[7] Valerio Perrone, Rodolphe Jenatton, Matthias W Seeger, and Cedric Archambeau. Scalable hyperparameter transfer learning. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems 31*, pages 6845–6855. Curran Associates, Inc., 2018.

[8] Chao Qin, Diego Klabjan, and Daniel Russo. Improving the expected improvement algorithm. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 5381–5391. Curran Associates, Inc., 2017.

[9] Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Mostofa Patwary, Mr Prabhat, and Ryan Adams. Scalable bayesian optimization using deep neural networks. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2171–2180, Lille, France, 07–09 Jul 2015. PMLR.

[10] Kevin Swersky, Jasper Snoek, and Ryan P. Adams. Multi-task bayesian optimization. In *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'13, pages 2004–2012, USA, 2013. Curran Associates Inc.